我说的都队

问径 408 搜题与答疑助手

数据库设计说明书

项目	内容
团队	我说的都队
课程	202601 软件工程
作业	原型设计与概要设计
日期	2026-11-07
代码基线	`echo14670/wenjing-408-agent`, `main@003b459`

1 引言

1.1 标识

本说明书适用于问径 408 题库及搜题与答疑助手，本次修订为 V1.5。题库关系化方案使用 SQLite 文件 `backend/data/wenjing.db`，由后端服务访问。

1.2 数据库概述

问径 408 面向 408 考研学生，提供题库检索、引导答疑、直接讲解和个人资料辅助学习。项目由课程团队作业提出，我说的都队负责开发与维护。

现有题库以 JavaScript 对象表示法（JSON）文件交付，后端启动时载入并建立检索索引；学习记录和个人资料保存在浏览器本地。本文给出 SQLite 题库关系化方案及其对象关系映射（ORM），作为后续实现依据，尚未接入当前运行代码。题库关系文件与后端部署在同一运行环境，浏览器通过业务接口访问。

主干 `main@003b459` 包含 16 道联调样例题。推荐开发分支 `feature/backend-recommend@9e19063` 包含 80 道交付题和推荐接口，80 题均为待核验。本文用该分支核对导入字段和推荐查询。

1.3 文档概述

本文按 GB/T 8567—2006 第 7.14 节《数据库（顶层）设计说明》的结构编排，依次说明数据库级设计决策、详细数据结构、访问软件和需求对应关系；实体－联系（ER）图、关系模式与 ORM 映射按课程要求展开。

说明书面向项目开发、测试、维护人员及课程评阅人员。用户会话、个人资料和模型调用密钥属于私有数据，存储与传输边界见第 3 章。

1.4 术语与缩写

名称	含义
Agent / LLM	智能体 / Large Language Model（大语言模型），用于答疑和讲解
BM25	题库检索使用的文本相关度评分方法
`context_chunks`	本次答疑选取的个人资料文本片段
DBDD / DBMS	Database Design Description / Database Management System，数据库设计说明 / 数据库管理系统
ER	Entity Relationship，实体 - 联系；ER 图描述对象、属性及联系
HTTP / API	Hypertext Transfer Protocol / Application Programming Interface，超文本传输协议 / 应用程序接口
JSON	JavaScript Object Notation，JavaScript 对象表示法
LocalStorage	浏览器本地键值存储，页面刷新后仍保留
MB	Megabyte，兆字节；本文按 1 MB = 1,000,000 字节估算
ORM	Object-Relational Mapping，对象关系映射
PK / FK / UK	Primary Key / Foreign Key / Unique Key，主键 / 外键 / 唯一约束
Pydantic	后端数据校验工具，负责题库载入和接口字段校验
SHA-256	Secure Hash Algorithm 256，256 位安全散列算法
SQL / DDL	Structured Query Language / Data Definition Language，结构化查询语言 / 数据定义语言

名称	含义
SQLAlchemy	Python ORM 工具，用于关系模型的实体映射
SQLite	单文件关系数据库，作为题库关系化方案的存储
UML	Unified Modeling Language，统一建模语言

2 引用文件

标识	文件	版本或日期	来源及用途
R01	课程团队作业（二）：原型设计与概要设计	2026-11-03 发布	课程作业页及团队整理件，确定设计交付范围
R02	GB/T 8567—2006《计算机软件文档编制规范》	2006-03-14 发布，第 7.14 节	国家标准全文公开系统，确定说明书结构
R03	问径 408 项目代码	`main@003b459`	项目仓库，现有题库、接口与本地存储基线
R04	题库接入与推荐分支	`9e19063`	同一仓库的 `feature/backend-recommend` 分支，提供 80 题字段设计输入
R05	题库设计、项目接口说明、接口契约	R03 对应版本	仓库 `docs/question-bank-design.md`、`docs/项目接口说明.md`、`docs/api-contract.md`
R06	《题库数据结构与检索方案设计（周玘晖）》	团队交付版，2026-10-29	团队共享资料，参考数据结构与存储方案
R07	《408搜题与答疑助手 - 资源来源表》	团队交付版	团队共享资料，提供题源、使用权限与核验信息
R08	《问径 UML 设计说明》与《统一接口与数据规范》	UML 1.2、第二轮交接	系统分层、运行类、后台原型和推荐字段

3 数据库级设计决策

3.1 数据分布与保存方式

系统的数据分布在后端题库、浏览器本地存储和请求处理过程。题库提供检索与核验依据；浏览器保存用户的学习过程；上传解析和模型调用产生的临时数据通过接口返回。



图 3-1 数据存储结构

实线表示现有数据流，虚线表示题库关系化后的读取路径。个人资料片段会随答疑请求临时发送，后端不将其写入资料库。

数据类别	主要内容	保存与管理方式
领域知识	题干、选项、标准答案、解析、科目、章节、考点	现有 `questions.json` 为题库来源，启动后载入内存并构建检索索引；关系化后按第 4 章表结构保存
来源与核验	出处、版权口径、核验状态、拟题与核验记录	现有题目保存 `source_ref` 和 `review_state`；关系模型补充来源表、关联表和核验记录表
用户交互记录	会话、消息、作答、思路、学习模式、答案揭示状态	保存于浏览器 `history:v1`，支持历史恢复、导入、导出和清空
个人资料	资料标题、类型、正文切片、创建时间	保存于 `materials:v1`，用户确认后写入；当前支持删除和清空
Agent 执行结果	模型回复、计算工具结果、引用和请求错误	后端通过 `reply/tool_result/citations` 返回；前端随会话保存消息、最近回复和错误状态
上传解析结果	`task_id`、提取文本、失败项	请求完成后返回前端；用户确认后将有效文本切片保存到本地，原文件和失败项不长期保存
运行数据	模型配置、检索阈值、内存索引	配置从环境变量读取；索引由题库重建；用户自带模型 Key 只在请求期间使用

3.2 数据库管理系统与运行环境

题库关系化方案使用支持 JSON 函数的 SQLite 3 和 SQLAlchemy 2.x，与现有 Python 后端保持一致；检索继续使用内存索引。查询时只读取当前生效版本，维护时通过受控流程导入、核验和发布数据。

项目	设计
数据库文件	`backend/data/wenjing.db`，后端本地 SQLite 单文件
运行与连接	与后端同机，由 SQLAlchemy 建立连接；每个连接开启 `PRAGMA foreign_keys=ON`
字符与格式	文本使用 Unicode；现有题库和导出文件使用 UTF-8；字段类型与格式见第 4.3 节
业务访问	前端通过 HTTP 接口访问；个人资料和会话继续保存在当前浏览器
查询组织	从 `active` 版本载入题目，构建内存检索索引；关系表索引支持题号、考点、来源和核验记录反查

3.3 输入、输出与业务规则

题库随版本发布更新，检索索引在启动或重新载入时重建。学习记录和个人资料保留到用户删除或清空；上传结果留在当前页面，离开页面后失效。

搜题结果分为 `original/similar/miss`，分别对应原题、相似题和未命中。检索服务确认题干身份和参数一致，并且题目已核验后，才绑定原题的 `question_id`。相似题只提供参考，未命中时会话题号可为空。用户输入的完整题干随会话保存。

引导模式通过 `answer_revealed` 记录用户是否主动查看答案，恢复会话时一并读取。题库标准答案、模型回复与工具核验结果分别保存、分别展示。`citations` 可包含已核验的题库来源及“用户提供的个人资料切片”，后者保留个人资料的来源身份。

3.4 容量、性能与访问控制

当前主干题库文件为 15,786 字节、16 题；80 题源文件为 82,853 字节。按该源文件量级估算，保留三个版本的题目正文约 0.25 MB，数据库页、关联记录与索引另计。容量随题数和保留版本数增长，发布时统计实际大小；该估算用于存储规划。

运行期以读取当前题库和内存检索为主。导入与版本切换按事务执行，候选索引在生效前构建；响应时间通过后续接口测试核对。

关系文件由后端运行账户访问，维护操作通过发布程序执行。浏览器不直接连接关系库，用户的会话和资料由浏览器存储隔离。用户自带模型 Key 只在请求期间使用，日志记录处理状态与数量，不保存密钥和资料正文。

3.5 数据更新、备份与恢复

3.5.1 版本发布与核验

发布依次完成导入、检查和切换，异常时恢复上一生效版本：

1. 导入草稿：转换题库字段、检查重复题号和枚举，在事务中写入 `draft` 版本及题目、关联数据；写入失败则回滚该事务。
2. 检查候选版本。 从草稿构建候选索引，核对题数、跳过数和外键。首次切换存储时，用同一题库快照分别从 JSON 和 SQLite 加载，对照题目字段、核验分布以及固定样例的检索三态和绑定题号。当前 `active` 版本在校验期间继续提供服务。
3. 切换版本：在同一事务中先将旧版本置为 `archived`，再将草稿置为 `active`；提交后切换运行时索引，核对 `/api/health` 的题数与索引数。

4. 处理发布后异常：通过新的事务恢复旧版本为 `active`，并恢复旧索引；新版本数据保留为归档，供后续排查。

核验对应草稿中的具体题目版本。维护程序在同一写事务中读取题目状态，追加记录并更新 `review_state`。核验记录保存原状态、新结论、执行人、时间与依据。拟题或历史导入缺少可靠前态时，`previous_state` 留空。题干或解析有实质改动时，重新核验后随新版本发布。

3.5.2 备份与清理

数据	备份与恢复	清理方式
现有题库文件	Git 保存版本，恢复后重新载入并构建索引	发布新文件时保留对应提交
SQLite 关系库	导入前保留一致性备份，使用 SQLite 备份接口或停机复制；恢复后检查外键与索引	归档旧版本，发布版本由维护流程限制物理删除
学习记录	用户导出本地历史 JSON，需要时重新导入	按会话删除或清空
个人资料	当前保存在本机浏览器，原始文件由用户保留	按资料删除或清空；清空后的内容需重新导入原文件

4 数据库详细设计

4.1 概念设计

4.1.1 题库实体

实体	标识	职责与联系
题库版本 QuestionBankVersion	`version_id`	管理发布状态，一个版本收录多道题目
题目 Question	`(bank_version_id, question_id)`	按版本保存题干、答案和解析，与考点、来源形成多对多联系
考点 KnowledgePoint	`kp_id`	保存考点字典，一个考点可关联多道题目
来源 Source	`source_id`	保存教材、课程等资源的出处和使用权限
核验记录 QuestionReview	`review_id`	记录某一版本题目的拟题、核验、争议与复核过程

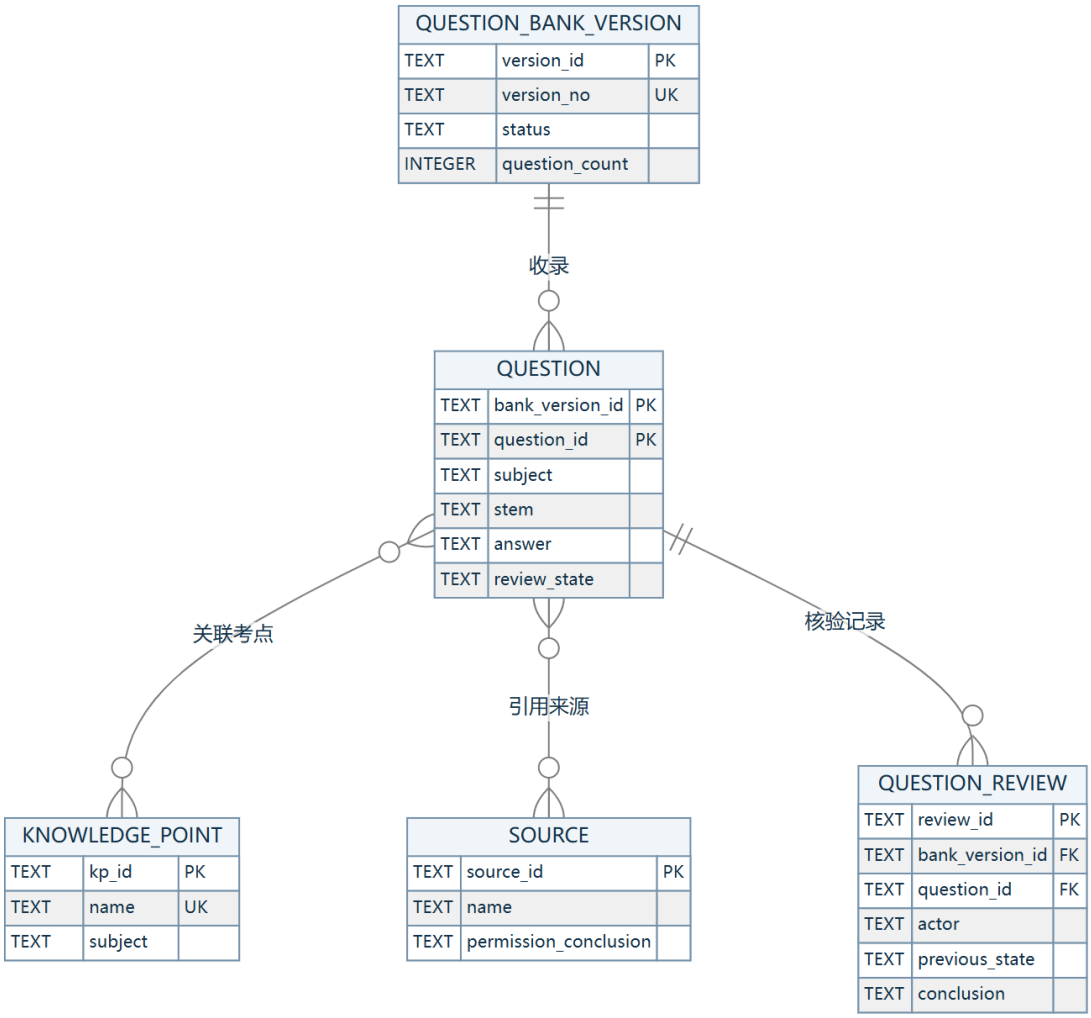


图 4-1 题库 ER 图

题目与考点通过 `question_knowledge_point` 关联，题目与来源通过 `question_source` 关联。两张关联表均携带版本 ID，使同一题号在不同版本中的考点、来源和核验信息各自对应。

题目以 `(bank_version_id, question_id)` 为复合主键。同一题号可以出现在多个版本中，旧版本内容保留；业务读取只使用 `active` 版本，对外接口仍使用稳定的 `question_id`。

4.1.2 浏览器本地对象

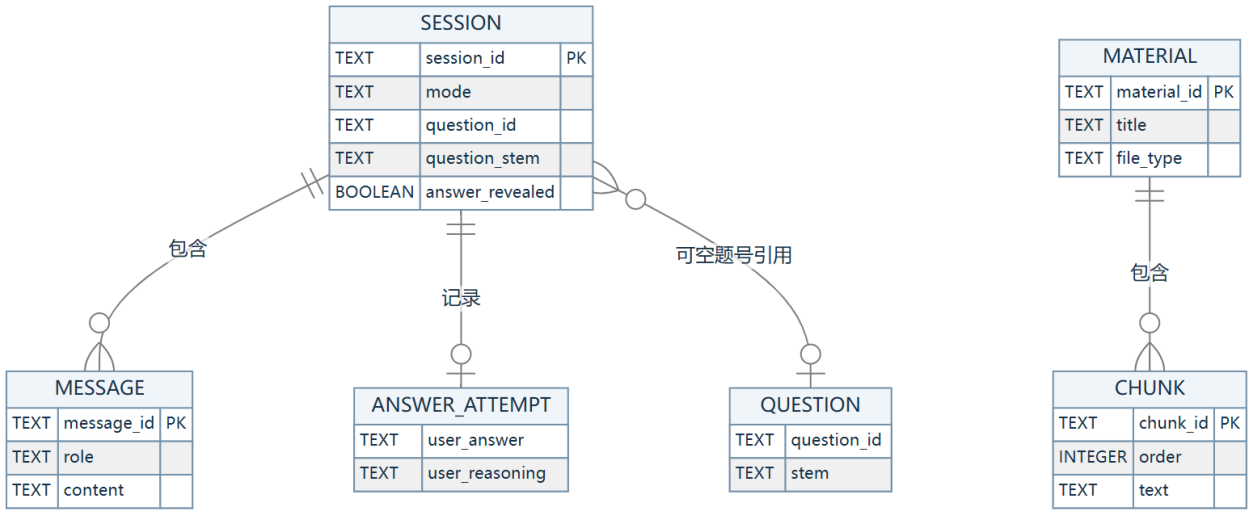


图 4-2 学习记录与个人资料 ER 图

会话和资料以本地 JSON 对象保存。图中的 `QUESTION` 来自后端题库，`Session.question_id` 是可空的业务引用，不建立跨浏览器与服务器的数据库外键。每个会话保留原始题干，刷新后可独立恢复当前题目。

4.2 逻辑结构与关系模式

题库关系化方案采用 SQLite，数据访问使用 SQLAlchemy 2.x。检索仍使用内存索引，关系库提供持久化数据与约束。

表	关系模式及用途
`question_bank_version`	`(version_id PK, version_no UK, updated_at, source_table, notes, question_count, skipped_count, checksum, commit_ref, status, created_at)`；题库发布记录
`question`	`(bank_version_id PK/FK, question_id PK, subject, chapter, question_type, stem, options_json, answer, solution, year, question_no, difficulty, permission, tags_json, source_ref, review_state, created_at, updated_at)`；题目版本快照
`knowledge_point`	`(kp_id PK, name UK, subject, created_at)`；考点字典
`question_knowledge_point`	`(bank_version_id PK/FK, question_id PK/FK, kp_id PK/FK)`；题目与考点关联
`source`	`(source_id PK, seq_no UK, group_name, name, edition, scope, resource_type, platform, url, permission_conclusion, permission_basis, usable_for_question, planned_kp, verifier, notes, created_at)`；资源来源
`question_source`	`(bank_version_id PK/FK, question_id PK/FK, source_id PK/FK, relation_note, is_primary)`；题目与来源关联
`question_review`	`(review_id PK, bank_version_id FK, question_id FK, action_type, actor, acted_at, previous_state, conclusion, comment, content_hash)`；核验过程

4.3 数据字典

以下各表与附录 A 的建表语句对应。类型中的长度标注用于字段设计；实际导入时同时做内容和枚举校验。

4.3.1 题库版本

字段	类型	键、可空与默认值	用途
`version_id`	TEXT(64)	PK；非空	题库版本的唯一标识
`version_no`	TEXT(32)	唯一；非空	版本号，如 `1.0`；与 JSON `version` 对应
`updated_at`	TEXT(32)	非空	题库数据更新日期（与 JSON `updated_at` 对应）
`source_table`	TEXT(256)	可空	来源资料表的文件标识或路径
`notes`	TEXT	可空	版本说明（对应 JSON `notes`）
`question_count`	INTEGER	非空；默认 `0`	该版本实际收录的有效题目数
`skipped_count`	INTEGER	非空；默认 `0`	导入时跳过的无效题目数
`checksum`	TEXT(64)	可空	题库文件 SHA-256，用于发布一致性校验
`commit_ref`	TEXT(64)	可空	对应代码提交
`status`	TEXT(16)	非空；默认 `'draft'`；draft / active / archived	版本状态；同一时刻至多一个 `active`（部分唯一索引保证）
`created_at`	TEXT(32)	非空；默认 `datetime('now')`	记录创建时间

4.3.2 题目

字段	类型	键、可空与默认值	用途
`bank_version_id`	TEXT(64)	PK；FK；非空	题目所属版本
`question_id`	TEXT(64)	PK；非空	稳定题号，同一版本内唯一
`subject`	TEXT(16)	非空	数据结构、计算机组成原理、操作系统、计算机网络
`chapter`	TEXT(128)	非空；默认 `''`	所属章节（推荐二级召回依据）
`question_type`	TEXT(20)	非空；默认 `'single_choice'`	题型，取值见第4.4节
`stem`	TEXT	非空	题干原文，检索主依据
`options_json`	TEXT	非空；默认 `'[]'`；有效JSON数组	选项数组（JSON），元素含选项标号；综合题为空数组
`answer`	TEXT	非空；默认 `''`	标准答案（如 `B`）
`solution`	TEXT	非空；默认 `''`	题库解析，与模型生成的回复分开保存
`year`	INTEGER	可空；1900-2100	有可靠出处时填写真题年份
`question_no`	INTEGER	可空；不小于1	有可靠出处时填写真题题号
`difficulty`	INTEGER	可空；1-5	难度等级；未评级时为空
`permission`	TEXT(16)	非空；默认 `'未标注'`；原创 / 变式 / 需变式 / 未标注	题目版权口径
`tags_json`	TEXT	非空；默认 `'[]'`；有效JSON数组	用于检索的关键词数组
`source_ref`	TEXT	非空；默认 `''`	题目出处原文；未匹配来源表时仍保留

字段	类型	键、可空与默认值	用途
`review_state`	TEXT(16)	非空；默认 `'pending'`；pending / verified / disputed	该版本题目的当前核验状态
`created_at`	TEXT(32)	非空；默认 `datetime('now')`	创建时间
`updated_at`	TEXT(32)	非空；默认 `datetime('now')`	更新时间（核验、内容修订时刷新）

4.3.3 考点

字段	类型	键、可空与默认值	用途
`kp_id`	TEXT(64)	PK；非空	稳定考点标识
`name`	TEXT(128)	唯一；非空	考点名称
`subject`	TEXT(16)	非空；默认 `''`	所属科目
`created_at`	TEXT(32)	非空；默认 `datetime('now')`	创建时间

4.3.4 题目与考点关联

字段	类型	键、可空与默认值	用途
`bank_version_id`	TEXT(64)	PK；FK；非空	复合主键之一；与 `question_id` 一起引用题目（ON DELETE CASCADE）
`question_id`	TEXT(64)	PK；FK；非空	复合主键之二
`kp_id`	TEXT(64)	PK；FK；非空	考点；被引用时不允许删除考点

4.3.5 资源来源

字段	类型	键、可空与默认值	用途
`source_id`	TEXT(64)	PK；非空	来源主键，建议 `SRC-001` 形式
`seq_no`	INTEGER	唯一；可空	资源来源表中的序号
`group_name`	TEXT(32)	非空	分组：四科或"项目过程方法参考"
`name`	TEXT(256)	非空	资源名称（如教材/课程名）
`edition`	TEXT(256)	可空	资源版本、书号或开课期次
`scope`	TEXT	可空	适用章节或内容范围
`resource_type`	TEXT(32)	可空	公开课、教材、教辅或配套习题集等
`platform`	TEXT(64)	可空	平台名称
`url`	TEXT(512)	可空	来源链接
`permission_conclusion`	TEXT	可空	使用权限结论（版权口径）
`permission_basis`	TEXT	可空	权限核实状态与依据（含"已查证/推断"）
`usable_for_question`	TEXT(16)	可空	是否可直接用于出题
`planned_kp`	TEXT	可空	计划覆盖的考点
`verifier`	TEXT(128)	可空	来源核验成员与日期
`notes`	TEXT	可空	备注

字段	类型	键、可空与默认值	用途
`created_at`	TEXT(32)	非空；默认 `datetime('now')`	创建时间

4.3.6 题目与来源关联

字段	类型	键、可空与默认值	用途
`bank_version_id`	TEXT(64)	PK；FK；非空	复合主键之一；与 `question_id` 引用题目（ON DELETE CASCADE）
`question_id`	TEXT(64)	PK；FK；非空	复合主键之二
`source_id`	TEXT(64)	PK；FK；非空	已匹配的来源；被引用时不允许删除来源
`relation_note`	TEXT	可空	引用关系与来源匹配依据
`is_primary`	INTEGER	非空；默认 `0`；0 / 1	是否为该版本题目的主要来源

4.3.7 核验记录

字段	类型	键、可空与默认值	用途
`review_id`	TEXT(64)	PK；非空	记录主键
`bank_version_id`	TEXT(64)	FK；非空	题目所属版本（与 `question_id` 组成复合外键）
`question_id`	TEXT(64)	FK；非空	被核验题目的题号，与版本ID共同定位题目
`action_type`	TEXT(16)	非空；propose / verify / dispute / reverify	拟题/核验/争议/复核
`actor`	TEXT(128)	非空	执行人（拟题人、核验人、复核人）
`acted_at`	TEXT(32)	非空	动作时间
`previous_state`	TEXT(16)	可空，状态枚举同下	核验前的题目状态，拟题或历史前态缺失时可为空
`conclusion`	TEXT(16)	非空，pending / verified / disputed	本次结论，同步写入 `question.review_state`
`comment`	TEXT	可空	保存核验依据及争议说明，提交核验时由维护程序校验
`content_hash`	TEXT(64)	可空	核验时题目内容的哈希，用于检查内容是否变化

4.4 物理结构、完整性与索引

关系文件的位置、连接方式与字符格式见第 3.2 节。表与索引的建表语句见附录 A。

项目	规则
主键	所有主键列非空；同一版本内题号唯一
题目引用	关联表、核验记录以 `(bank_version_id, question_id)` 引用题目
发布状态	`draft` 为草稿、`active` 为当前版本、`archived` 为归档；部分唯一索引保证至多一个 `active`
主来源	同一版本的一道题至多一个 `is_primary=1` 的来源

项目	规则
核验状态	`pending / verified / disputed`，核验时在同一事务中记录前后状态并更新题目
题型	`single_choice / multiple_choice / fill_blank / short_answer / calculation / comprehensive`
数组字段	`options_json` 保留选项顺序，`tags_json` 保存标签；导入层检查数组及元素类型
删除	删除草稿题目时，关联和核验记录级联清理；被引用的考点、来源使用 `RESTRICT`
历史保留	已发布版本和题目由发布维护流程限制修改与删除，内容修订通过新版本发布

外键在每个 SQLite 连接上开启 `PRAGMA foreign_keys=ON`。科目、章节、核验状态、题号及版本字段建立普通索引；两张关联表按考点和来源建立反查索引。检索索引继续在内存中构建。

考点、来源和核验记录分别管理，减少重复内容。`review_state` 保存在题目中以支持检索判定，核验记录保留过程；`source_ref` 保留原始出处，便于兼容现有题库。选项与标签属于题目的小集合，使用 JSON 字段保存。

4.5 浏览器本地数据结构

对象	主要字段	保存与恢复
Session	`session_id, mode, question_id, question_stem, match_type, answer_revealed, title, created_at, updated_at`	`history:v1` 数组中的一个会话
Message	`message_id/id, role, content, model_source, kind, citations, created_at/createdAt`	会话内 `messages[]`；适配层兼容现有两种标识和时间字段
AnswerAttempt	`user_answer, user_reasoning`	会话内 `answer_attempt`；当前调用主要使用 `user_answer`
Material	`material_id, title, file_type, chunks, created_at, updated_at`	`materials:v1` 数组中的一份资料
Chunk	`chunk_id, order, text`	资料内 `chunks[]`，正文按顺序恢复
结果快照	`result_snapshot, last_reply, chat_status, chat_error`	随会话保存检索结果、最近回复和重试所需状态

`last_reply` 包含 `reply`、`toolResult` 和 `citations`；消息保留每轮回复正文与引用。旧页面将工具结果附在消息上，新页面适配层将最近一次工具结果放入 `last_reply.toolResult`。这些数据用于页面恢复，不进入服务端数据库。

个人资料正文存放在切片中，按 `maxLength=800`、`overlap=80`、`minChunk=120` 切分；答疑再按当前问题选取片段生成 `context_chunks`。资料切片长期保存在浏览器，选片结果仅用于本次请求。

历史导出结构为 `{version:1, exported_at, sessions:[]}`，导入时按 `session_id` 去重。个人资料当前提供删除、清空，文件导入导出属于后续开发内容。

5 用于数据库访问或操纵的软件配置项的详细设计

5.1 访问模块、输入与输出

下表说明现有访问模块及关系化后的衔接。关系库读取替换文件读取，加载层对检索与答疑提供的题目对象保持一致。

模块	输入	访问与处理	输出
`QuestionStore.load/reload`	题库来源或当前生效版本	当前校验 JSON；关系化后读取 `active` 版本，转换为题目列表	`Question[]`、载入统计与跳过原因
`QuestionStore.get_question`	`question_id`	从当前加载的题库中查找题目	一个 `Question` 或空值
`QuestionStore.stats`	当前题库状态	汇总题数、科目与核验分布、跳过记录	健康检查使用的统计对象
检索与答疑接口	题干、题号、会话上下文、模式、资料片段	读取题库与索引，执行检索、模型调用或工具计算	三态检索结果，或 `reply/tool_result/citations`
上传解析模块	文件字节与文件类型	提取文本并收集失败项	`task_id/extracted_text/failure_details`
前端历史与资料存储	会话、消息或确认后的资料正文	将业务对象序列化到本地存储，恢复时反序列化	本地会话与资料对象
版本维护程序（关系化方案）	版本元数据、题目、考点、来源和核验记录	事务导入、候选检查、版本切换或回退	生效版本、发布统计和错误记录

接口请求的字段与数量上限由现有校验模型检查。导入时验证主键、外键和枚举，失败则回滚当前写入事务；当前生效版本继续提供查询。前端保留答疑错误状态与重试所需输入，具体数据流见第 5.5 节。

5.2 实体类与表的 ORM 映射

业务实体	ORM 类	对应表	关联
题库版本	QuestionBankVersion	`question_bank_version`	`questions`：一对多题目
题目	Question	`question`	复合主键；关联考点、来源与核验记录
考点	KnowledgePoint	`knowledge_point`	通过关联表与题目形成多对多
题目与考点	关联表或关联类	`question_knowledge_point`	三列联合主键，复合外键引用题目
来源	Source	`source`	通过 QuestionSource 关联多道题目
题目与来源	QuestionSource	`question_source`	关联对象保存 `relation_note/is_primary`
核验记录	QuestionReview	`question_review`	复合外键引用题目版本，`previous_state` 和 `conclusion` 分别保存前后状态

SQLAlchemy 实体负责持久化与关联查询，现有 Pydantic 模型负责题库载入和 HTTP 接口校验。`Session/Message/Material/Chunk` 由前端对象与本地 JSON 映射，继续使用第 4.5 节的存储结构。

5.3 ORM 映射示例

下面展示版本、题目、考点及关联表的关键字段。完整字段和约束见第 4 章与附录 A。

```
from __future__ import annotations
from sqlalchemy import Column, ForeignKey, ForeignKeyConstraint
from sqlalchemy import String, Table, Text
from sqlalchemy.orm import DeclarativeBase, Mapped
from sqlalchemy.orm import mapped_column, relationship
```

```
class Base(DeclarativeBase):
    pass

question_knowledge_point = Table(
    "question_knowledge_point", Base.metadata,
    Column("bank_version_id", String(64),
           primary_key=True, nullable=False),
    Column("question_id", String(64),
           primary_key=True, nullable=False),
    Column("kp_id", String(64),
           primary_key=True, nullable=False),
    ForeignKeyConstraint(
        ["bank_version_id", "question_id"],
        ["question.bank_version_id", "question.question_id"],
        ondelete="CASCADE",
    ),
    ForeignKeyConstraint(
        ["kp_id"], ["knowledge_point.kp_id"],
        ondelete="RESTRICT",
    ),
)

class QuestionBankVersion(Base):
    __tablename__ = "question_bank_version"
    version_id: Mapped[str] = mapped_column(
        String(64), primary_key=True)
    version_no: Mapped[str] = mapped_column(
        String(32), unique=True)
    questions: Mapped[list["Question"]] = relationship(
        back_populates="bank_version")

class Question(Base):
    __tablename__ = "question"
    bank_version_id: Mapped[str] = mapped_column(
        String(64),
        ForeignKey("question_bank_version.version_id",
                   ondelete="RESTRICT"),
        primary_key=True,
    )
    question_id: Mapped[str] = mapped_column(
        String(64), primary_key=True)
    subject: Mapped[str] = mapped_column(String(16))
    stem: Mapped[str] = mapped_column(Text)
    bank_version: Mapped["QuestionBankVersion"] = relationship(
        back_populates="questions")
    knowledge_points: Mapped[list["KnowledgePoint"]] = relationship(
        secondary=question_knowledge_point,
        back_populates="questions")

class KnowledgePoint(Base):
    __tablename__ = "knowledge_point"
    kp_id: Mapped[str] = mapped_column(
        String(64), primary_key=True)
    name: Mapped[str] = mapped_column(
        String(128), unique=True)
    questions: Mapped[list["Question"]] = relationship(
        secondary=question_knowledge_point,
        back_populates="knowledge_points")
```

5.4 现有数据的字段映射

输入字段	目标字段或对象	处理方式
`question_id`	`Question.question_id`	保持题号，导入时同时写入所属版本
`options` 对象或数组	`options_json`	对象按选项标号排序后转为有序数组
`analysis / solution`	`solution`	源题库的 `analysis` 映射为内部解析字段
`type / question_type`	`question_type`	统一为第 4.4 节题型枚举
`knowledge_point / knowledge_points`	`knowledge_point` 与关联表	单值转数组，按统一考点字典匹配稳定 ID
`source_ref`	`source_ref` 与 `question_source`	保留原文，人工确认匹配后建立来源关联
`review_state`	`review_state`	中文状态归一为 `pending/verified/disputed`
`reviewer`	`QuestionReview.actor`	源文件中表示拟题人，写入 `propose` 记录
核验原型 `at`、`by`、`from`、`to`	`acted_at`、`actor`、`previous_state`、`conclusion`	记录动作时间、执行人和前后状态
核验原型 `basis/note`	`QuestionReview.comment`	保存核验依据，涉及争议时附说明
`version/updated_at/source_table/meta`	`QuestionBankVersion`	保存版本信息和导入统计

未匹配的来源保留原文，迁移报告记录题号、出处和原因。80 题交付件中考点名称尚未复用，数据负责人按统一字典归并后建立题目与考点关联。拟题与核验分别记录，核验人员、时间和结论按实际结果填写。

5.5 业务数据流与错误处理

业务	数据经过的模块	保存结果
搜题	原始题干 → `/api/search` → 题库加载层与检索索引 → 三态结果	原始题干、题目身份及检索快照进入本地会话
答疑	当前题干、最近对话、作答和选定切片 → `/api/chat` → Agent、模型与计算工具 → 回复及引用	消息、最近回复、揭示状态和错误状态进入本地会话
资料处理	文件 → `/api/upload` → 文本与失败项 → 前端预览、修正、确认 → 切片	有效正文保存为本地资料；答疑时选片发送
历史恢复	选择记录或页面刷新 → 按会话 ID 与题干恢复 → 还原模式、对话和揭示状态	沿用该题自己的记录，新题另建会话
推荐练习（开发分支）	`/api/recommend` 接收当前题号和浏览器提供的已做题号，按考点或章节查找候选	返回 `questions/scope/notice`，浏览器为新题建立独立会话

未命中且没有个人资料时，Agent 返回资料不足说明；模型调用失败时返回错误，前端保留会话供重试。上传部分成功时展示成功文本和失败项，用户确认后保存有效内容。

5.5.1 关系化后的推荐候选查询

推荐分支通过内存索引召回。题库关系化后，在同一读事务中读取生效版本、当前题目和候选题。默认查询筛选已核验题，按共享考点数量排序，并排除当前题与前端传来的已做题。`done_ids_json` 由 `done_question_ids` 转成 JSON 数组，缺省为 `[]`。

```
SELECT q.question_id
FROM question_knowledge_point AS own
JOIN question_knowledge_point AS candidate
  ON candidate.bank_version_id = own.bank_version_id
 AND candidate.kp_id = own.kp_id
JOIN question AS q
  ON q.bank_version_id = candidate.bank_version_id
 AND q.question_id = candidate.question_id
WHERE own.bank_version_id = :active_version
  AND own.question_id = :current_id
  AND q.question_id <> :current_id
  AND q.review_state = 'verified'
  AND NOT EXISTS (
    SELECT 1 FROM json_each(:done_ids_json) AS done
    WHERE done.value = q.question_id
  )
GROUP BY q.question_id
ORDER BY COUNT(*) DESC, q.question_id
LIMIT :top_k;
```

同考点候选为空时，按当前题目的科目和非空章节查询，沿用版本、核验及排除条件。前端根据 `scope` 区分同考点与同章节练习。两级都没有候选时返回空列表和 `scope=none`。

5.6 与系统架构和类图的对应

系统模块	UML 图	数据设计对应
题库加载与检索	01、07	`QuestionStore` 提供 `Question`，检索服务据此构建内存索引。关系化后按生效版本还原题目对象。
Agent 答疑	05、07	`AgentContext` 和 `CalculationOutcome` 用于单次请求。返回的回复、计算结果和引用随本地会话管理。
学习记录与资料	10、13	`LearningSession` 对应本说明书的 `Session`。消息、作答、资料与切片使用浏览器存储，个人记忆属于后续设计。
后台核验与导入	14、16、17	原型核验记录对应 `QuestionReview` 的状态流转字段。导入批次和行级异常用于处理与预览，题库发布版本与统计由 `QuestionBankVersion` 保存。

数据加载层将关系记录还原为现有题目对象，检索、答疑和资料解析接口按现行契约传输数据。

6 需求的可追踪性

DB01—DB08 是本文的数据需求索引。下表将需求与数据对象、访问模块及设计章节关联；也可按数据对象反向查找其对应需求。

编号	数据需求	对应对象与访问模块	设计位置
DB01	保存和检索领域知识	`question_bank_version/question/knowledge_point/question_knowledge_point`；QuestionStore 与检索模块	3.1、4.1—4.4、5.1—5.4

编号	数据需求	对应对象与访问模块	设计位置
DB02	管理出处、版权及核验过程	`source/question_source/question_review` 和题目核验状态；版本维护程序	3.3、3.5、4.3、5.4
DB03	保存用户交互与学习记录	Session、Message、AnswerAttempt、检索快照；前端历史存储	3.1、4.1.2、4.5、5.5
DB04	保存 Agent 或系统任务结果	`last_reply`、消息、工具结果、引用和错误状态；答疑接口与前端存储	3.1、4.5、5.1、5.5
DB05	管理个人资料及答疑片段	Material、Chunk、`context_chunks`；资料存储与选片模块	3.1、4.1.2、4.5、5.5
DB06	处理上传成功、部分失败与确认保存	UploadResponse、`failure_details`；上传模块与前端资料存储	3.1、5.1、5.5
DB07	支持题库版本更新、核验和回退	版本表、题目及按版本保存的关联与核验记录；版本维护程序	3.5、4.2—4.4、附录 A
DB08	管理运行数据并保护私有数据	环境配置、运行索引、模型 Key；后端运行环境与本地存储	3.2—3.4、4.5、5.1

ER 图见第 4.1 节，关系模式和字段设计见第 4.2—4.4 节，ORM 映射见第 5.2—5.4 节。第 5.6 节给出与系统架构和类图命名的对应。

7 注解

术语与缩写已在第 1.4 节提前定义。表名和数据库字段使用下划线命名；前端适配层的字段沿用现有命名，并在第 4.5、5.4 节说明映射。

关系模式中多个字段同时标记 PK 表示共同构成主键；FK 表示外键，其引用范围和删除规则见第 4.4 节及附录 A。题目由版本 ID 与稳定题号共同定位，对外接口仍使用题号，读取时限定当前生效版本。

字段字典中的长度为设计取值范围，导入层按相同约定校验；选项与标签的 JSON 字段还需检查数组与元素类型。

附录 A 建表语句

以下语句对应第 4 章的关系模型。SQLite 连接启用外键校验，并提供 JSON 函数。

```
PRAGMA foreign_keys = ON;

CREATE TABLE question_bank_version (
    version_id      TEXT(64)    NOT NULL PRIMARY KEY,
    version_no      TEXT(32)    NOT NULL UNIQUE,
    updated_at      TEXT(32)    NOT NULL,
    source_table    TEXT(256),
    notes           TEXT,
    question_count  INTEGER     NOT NULL DEFAULT 0 CHECK (question_count >= 0),
    skipped_count   INTEGER     NOT NULL DEFAULT 0 CHECK (skipped_count >= 0),
    checksum        TEXT(64),
    commit_ref      TEXT(64),
    status          TEXT(16)    NOT NULL DEFAULT 'draft'
                                CHECK (status IN ('draft','active','archived')),
    created_at      TEXT(32)    NOT NULL DEFAULT (datetime('now'))
);
CREATE UNIQUE INDEX uq_bank_active
```

```
    ON question_bank_version(status) WHERE status = 'active';

CREATE TABLE question (
    bank_version_id  TEXT(64)   NOT NULL,
    question_id      TEXT(64)   NOT NULL,
    subject          TEXT(16)   NOT NULL
                                CHECK (subject IN ('数据结构','计算机组成原理','操作系统','计算机网络')),
    chapter          TEXT(128)  NOT NULL DEFAULT '',
    question_type    TEXT(20)   NOT NULL DEFAULT 'single_choice'
                                CHECK (question_type IN ('single_choice','multiple_choice',
                                       'fill_blank','short_answer','calculation','comprehensive')),
    stem             TEXT       NOT NULL CHECK (length(trim(stem)) > 0),
    options_json     TEXT       NOT NULL DEFAULT '[]' CHECK (json_valid(options_json)),
    answer           TEXT       NOT NULL DEFAULT '',
    solution         TEXT       NOT NULL DEFAULT '',
    year             INTEGER    CHECK (year IS NULL OR (year BETWEEN 1900 AND 2100)),
    question_no      INTEGER    CHECK (question_no IS NULL OR question_no >= 1),
    difficulty       INTEGER    CHECK (difficulty IS NULL OR (difficulty BETWEEN 1 AND 5)),
    permission       TEXT(16)   NOT NULL DEFAULT '未标注'
                                CHECK (permission IN ('原创','变式','需变式','未标注')),
    tags_json        TEXT       NOT NULL DEFAULT '[]' CHECK (json_valid(tags_json)),
    source_ref       TEXT       NOT NULL DEFAULT '',
    review_state     TEXT(16)   NOT NULL DEFAULT 'pending'
                                CHECK (review_state IN ('pending','verified','disputed')),
    created_at       TEXT(32)   NOT NULL DEFAULT (datetime('now')),
    updated_at       TEXT(32)   NOT NULL DEFAULT (datetime('now')),
    PRIMARY KEY (bank_version_id, question_id),
    FOREIGN KEY (bank_version_id)
        REFERENCES question_bank_version(version_id) ON DELETE RESTRICT
);
CREATE INDEX idx_question_question_id   ON question(question_id);
CREATE INDEX idx_question_subject       ON question(subject);
CREATE INDEX idx_question_review_state  ON question(review_state);
CREATE INDEX idx_question_chapter       ON question(chapter);
CREATE INDEX idx_question_bank_version  ON question(bank_version_id);
CREATE INDEX idx_question_question_type ON question(question_type);

CREATE TABLE knowledge_point (
    kp_id       TEXT(64)   NOT NULL PRIMARY KEY,
    name        TEXT(128)  NOT NULL UNIQUE,
    subject     TEXT(16)   NOT NULL DEFAULT ''
                           CHECK (subject = '' OR subject IN ('数据结构','计算机组成原理','操作系统','计算机网
络')),
    created_at  TEXT(32)   NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE question_knowledge_point (
    bank_version_id  TEXT(64) NOT NULL,
    question_id      TEXT(64) NOT NULL,
    kp_id            TEXT(64) NOT NULL,
    PRIMARY KEY (bank_version_id, question_id, kp_id),
    FOREIGN KEY (bank_version_id, question_id)
        REFERENCES question(bank_version_id, question_id) ON DELETE CASCADE,
    FOREIGN KEY (kp_id)
        REFERENCES knowledge_point(kp_id) ON DELETE RESTRICT
);
CREATE INDEX idx_qkp_kp ON question_knowledge_point(kp_id);

CREATE TABLE source (
    source_id             TEXT(64)   NOT NULL PRIMARY KEY,
    seq_no                INTEGER    UNIQUE,
    group_name            TEXT(32)   NOT NULL,
```

```
    name                  TEXT(256)  NOT NULL,
    edition               TEXT(256),
    scope                 TEXT,
    resource_type         TEXT(32),
    platform              TEXT(64),
    url                   TEXT(512),
    permission_conclusion TEXT,
    permission_basis      TEXT,
    usable_for_question   TEXT(16)
                          CHECK (usable_for_question IS NULL OR usable_for_question IN
                                 ('可直接使用','需变式','仅参考','需原创','待确认')),
    planned_kp            TEXT,
    verifier              TEXT(128),
    notes                 TEXT,
    created_at            TEXT(32)   NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_source_group ON source(group_name);

CREATE TABLE question_source (
    bank_version_id  TEXT(64) NOT NULL,
    question_id      TEXT(64) NOT NULL,
    source_id        TEXT(64) NOT NULL,
    relation_note    TEXT,
    is_primary       INTEGER  NOT NULL DEFAULT 0 CHECK (is_primary IN (0,1)),
    PRIMARY KEY (bank_version_id, question_id, source_id),
    FOREIGN KEY (bank_version_id, question_id)
        REFERENCES question(bank_version_id, question_id) ON DELETE CASCADE,
    FOREIGN KEY (source_id) REFERENCES source(source_id) ON DELETE RESTRICT
);
CREATE INDEX idx_qs_source ON question_source(source_id);
CREATE UNIQUE INDEX uq_qs_primary
    ON question_source(bank_version_id, question_id) WHERE is_primary = 1;

CREATE TABLE question_review (
    review_id        TEXT(64)  NOT NULL PRIMARY KEY,
    bank_version_id  TEXT(64)  NOT NULL,
    question_id      TEXT(64)  NOT NULL,
    action_type      TEXT(16)  NOT NULL
                               CHECK (action_type IN ('propose','verify','dispute','reverify')),
    actor            TEXT(128) NOT NULL,
    acted_at         TEXT(32)  NOT NULL,
    previous_state   TEXT(16)
                               CHECK (previous_state IS NULL OR
                                      previous_state IN ('pending','verified','disputed')),
    conclusion       TEXT(16)  NOT NULL
                               CHECK (conclusion IN ('pending','verified','disputed')),
    comment          TEXT,
    content_hash     TEXT(64),
    FOREIGN KEY (bank_version_id, question_id)
        REFERENCES question(bank_version_id, question_id) ON DELETE CASCADE
);
CREATE INDEX idx_review_question ON question_review(bank_version_id, question_id);
CREATE INDEX idx_review_acted_at ON question_review(acted_at);
```